

ASSIGNMENT

High Performance and Cloud Computing

Course Code : 23AID304

Submitted by

Kundurthi Abhiram

DL.AI.U4AID24144

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY (B.Tech)

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE (AIDS)

Semester V

Course Professor : Prof. Suraj Singh



AMRITA VISHWA VIDYAPEETHAM

School of Artificial Intelligence

Faridabad, Delhi NCR

September 2026

ASSIGNMENT

QUESTION

Explain the common OpenMP routines (functions) and their uses.

ANSWER

OpenMP provides a set of library routines that allow programmers to control and obtain information about parallel execution. These routines are declared in the `omp.h` header file. They are commonly used for controlling the number of threads, identifying threads, checking the parallel execution environment, and measuring execution time.

1. `omp_get_thread_num()`

The `omp_get_thread_num()` function returns the identification number of the thread that is currently executing. Thread numbering normally starts from 0.

For example, if four threads are executing, their thread IDs will generally be 0, 1, 2, and 3.

```
#include <stdio.h>
#include <omp.h>

int main()
{
    #pragma omp parallel
    {
        printf("Thread ID = %d\n", omp_get_thread_num());
    }

    return 0;
}
```

This function is useful when a program needs to identify which thread is performing a particular operation.

2. `omp_get_num_threads()`

The `omp_get_num_threads()` function returns the number of threads currently participating in a parallel region.

```
#include <stdio.h>
#include <omp.h>

int main()
{
```

```
#pragma omp parallel
{
    printf("Total threads = %d\n",
           omp_get_num_threads());
}

return 0;
}
```

If four threads are executing, each thread can obtain the value 4 from this function.

It is important to note that when this function is called outside a parallel region, it normally returns 1 because only the initial thread is executing.

3. `omp_set_num_threads(n)`

The `omp_set_num_threads()` function is used to request a specific number of threads for subsequent parallel regions.

```
#include <stdio.h>
#include <omp.h>

int main()
{
    omp_set_num_threads(4);

    #pragma omp parallel
    {
        printf("Thread ID = %d\n",
               omp_get_thread_num());
    }

    return 0;
}
```

In this example, the program requests four threads for the parallel region.

This function is useful when the programmer wants to control the number of threads used by OpenMP.

4. `omp_get_max_threads()`

The `omp_get_max_threads()` function returns the maximum number of threads that OpenMP would use for a parallel region when no other setting overrides the value.

```
#include <stdio.h>
```

```
#include <omp.h>

int main()
{
    printf("Maximum threads = %d\n",
           omp_get_max_threads());

    return 0;
}
```

This function is useful for determining the default or maximum thread count available for parallel execution.

5. `omp_get_wtime()`

The `omp_get_wtime()` function returns the elapsed wall-clock time. It is commonly used to measure the execution time of sequential and parallel programs.

```
#include <stdio.h>
#include <omp.h>

int main()
{
    double start, end;

    start = omp_get_wtime();

    #pragma omp parallel
    {
        /* Parallel work */
    }

    end = omp_get_wtime();

    printf("Execution time = %f seconds\n",
           end - start);

    return 0;
}
```

The difference between the ending time and starting time gives the approximate execution time of the measured section.

6. `omp_get_wtick()`

The `omp_get_wtick()` function returns the timer resolution, or the precision of the timer used by `omp_get_wtime()`.

```
#include <stdio.h>
#include <omp.h>

int main()
{
    printf("Timer resolution = %f\n",
           omp_get_wtick());

    return 0;
}
```

It is useful when analyzing the precision of execution-time measurements.

7. `omp_in_parallel()`

The `omp_in_parallel()` function determines whether the current code is executing inside a parallel region.

It returns a non-zero value when execution is inside a parallel region and returns zero when execution is not inside a parallel region.

```
#include <stdio.h>
#include <omp.h>

int main()
{
    printf("Outside parallel region: %d\n",
           omp_in_parallel());

    #pragma omp parallel
    {
        printf("Inside parallel region: %d\n",
               omp_in_parallel());
    }

    return 0;
}
```

This function is useful for determining the current parallel execution state of the program.

8. `omp_get_num_procs()`

The `omp_get_num_procs()` function returns the number of processors available to the OpenMP program.

```
#include <stdio.h>
#include <omp.h>

int main()
{
    printf("Available processors = %d\n",
        omp_get_num_procs());

    return 0;
}
```

This function provides information about the processing resources available to the program. The number of available processors does not necessarily equal the number of threads currently executing.

Summary of Common OpenMP Routines

OpenMP Routine	Use
<code>omp_get_thread_num()</code>	Returns the ID of the current thread.
<code>omp_get_num_threads()</code>	Returns the number of threads in the current parallel region.
<code>omp_set_num_threads(n)</code>	Sets the requested number of threads for subsequent parallel regions.
<code>omp_get_max_threads()</code>	Returns the maximum/default number of threads for a parallel region.
<code>omp_get_wtime()</code>	Returns wall-clock time and is used for measuring execution time.
<code>omp_get_wtick()</code>	Returns the timer resolution.
<code>omp_in_parallel()</code>	Checks whether execution is currently inside a parallel region.
<code>omp_get_num_procs()</code>	Returns the number of processors available to the program.

CONCLUSION

OpenMP library routines provide essential control and information for shared-memory parallel programming. Functions such as `omp_get_thread_num()` and `omp_get_num_threads()` help identify and count threads, while `omp_set_num_threads()` provides control over the requested number of threads. Timing routines such as `omp_get_wtime()` are useful for

evaluating the performance of parallel programs. Together, these routines help programmers understand, control, and measure OpenMP-based parallel execution.